

❑ 50 PRINCIPAIS COMANDOS DO PY AUTOGUI PARA RPA

❑ CATEGORIAS DOS COMANDOS:

1. Controle do Mouse
2. Controle do Teclado
3. Captura de Tela e Reconhecimento
4. Alertas e Mensagens
5. Configurações e Controle
6. Funções Avançadas para RPA
7. Tratamento de Erros e Segurança

❑ 50 COMANDOS ESSENCIAIS DO PYAUTOGUI

1. CONTROLE DO MOUSE

python

```
import pyautogui
import time

# CONFIGURAÇÃO INICIAL (IMPORTANTE!)
pyautogui.FAILSAFE = True # Move mouse para canto superior esquerdo para abortar
pyautogui.PAUSE = 0.5 # Pausa entre comandos (segundos)

# 1. Obter posição atual do mouse
x, y = pyautogui.position()
print(f"Posição atual: X={x}, Y={y}")

# 2. Mover mouse para coordenadas absolutas
pyautogui.moveTo(100, 200) # X, Y
pyautogui.moveTo(x=500, y=300, duration=1) # Com animação (1 segundo)
```

```

# 3. Mover mouse relativo à posição atual
pyautogui.moveRel(50, 0) # Move 50 pixels para direita
pyautogui.moveRel(0, -100) # Move 100 pixels para cima

# 4. Clicar em posição específica
pyautogui.click(x=100, y=200) # Clica em coordenadas
pyautogui.click() # Clica na posição atual

# 5. Tipos de clique
pyautogui.doubleClick() # Duplo clique
pyautogui.rightClick() # Clique direito
pyautogui.middleClick() # Clique do meio (scroll)
pyautogui.tripleClick() # Triplo clique

# 6. Clicar e arrastar (drag)
pyautogui.dragTo(500, 400, duration=1) # Arrastar para posição
pyautogui.dragRel(100, 0, duration=0.5) # Arrastar relativo

# 7. Pressionar e soltar mouse
pyautogui.mouseDown() # Pressiona botão
pyautogui.mouseUp() # Solta botão

# 8. Scroll (roda do mouse)
pyautogui.scroll(10) # Scroll para cima
pyautogui.scroll(-20) # Scroll para baixo

# 9. HScroll (scroll horizontal - se suportado)
pyautogui.hscroll(10) # Direita
pyautogui.hscroll(-10) # Esquerda

# 10. Velocidade do mouse
pyautogui.MINIMUM_DURATION = 0.1 # Duração mínima para movimentos
pyautogui.MINIMUM_SLEEP = 0.05 # Tempo mínimo entre movimentos

```

2. CONTROLE DO TECLADO

```
python
```

```

# 11. Escrever texto
pyautogui.write('Hello World!', interval=0.1) # Intervalo entre teclas

# 12. Pressionar tecla individual

```

```
pyautogui.press('enter')
pyautogui.press('tab')
pyautogui.press('esc')
pyautogui.press('f5')

# 13. Pressionar múltiplas teclas
pyautogui.hotkey('ctrl', 'c')      # Ctrl + C (copiar)
pyautogui.hotkey('ctrl', 'v')      # Ctrl + V (colar)
pyautogui.hotkey('ctrl', 'a')      # Ctrl + A (selecionar tudo)
pyautogui.hotkey('ctrl', 's')      # Ctrl + S (salvar)
pyautogui.hotkey('alt', 'tab')      # Alt + Tab (alternar janelas)
pyautogui.hotkey('winleft', 'd')    # Windows + D (área de trabalho)

# 14. Pressionar e segurar teclas
pyautogui.keyDown('shift')          # Pressiona Shift
pyautogui.keyUp('shift')            # Solta Shift

# 15. Lista de teclas especiais
# 'enter', 'tab', 'esc', 'backspace', 'delete'
# 'home', 'end', 'pageup', 'pagedown'
# 'up', 'down', 'left', 'right'
# 'f1' a 'f12'
# 'winleft', 'winright', 'alt', 'ctrl', 'shift'

# 16. Teclas especiais para Windows/Linux/Mac
pyautogui.press('winleft')          # Tecla Windows
pyautogui.press('command')          # Tecla Command (Mac)

# 17. Limpar campo (Backspace múltiplo)
pyautogui.press(['backspace'] * 10) # 10 backspaces

# 18. Teclas de função
pyautogui.press('f11')              # Tela cheia
pyautogui.press('f5')              # Atualizar

# 19. Navegação em texto
pyautogui.hotkey('ctrl', 'home')     # Início do documento
pyautogui.hotkey('ctrl', 'end')      # Fim do documento
pyautogui.hotkey('ctrl', 'left')     # Palavra à esquerda
pyautogui.hotkey('ctrl', 'right')    # Palavra à direita
```

3. CAPTURA DE TELA E RECONHECIMENTO

python

```
# 20. Capturar tela inteira
screenshot = pyautogui.screenshot()
screenshot.save('screenshot.png')

# 21. Capturar região específica
region = pyautogui.screenshot(region=(0, 0, 300, 400)) # X, Y, Width, Height

# 22. Obter cor de um pixel
pixel_color = pyautogui.pixel(100, 200) # Retorna (R, G, B)
print(f"Cor RGB: {pixel_color}")

# 23. Verificar se cor corresponde
matches = pyautogui.pixelMatchesColor(100, 200, (255, 255, 255))
matches_tolerance = pyautogui.pixelMatchesColor(100, 200, (255, 255, 255),
tolerance=10)

# 24. Localizar imagem na tela
try:
    button_location = pyautogui.locateOnScreen('botao.png')
    print(f"Imagem encontrada em: {button_location}")
except pyautogui.ImageNotFoundException:
    print("Imagem não encontrada")

# 25. Localizar todas as ocorrências
buttons = pyautogui.locateAllOnScreen('botao.png')
for button in buttons:
    print(button)

# 26. Localizar com confiança (tolerância)
button = pyautogui.locateOnScreen('botao.png', confidence=0.8) # 80% de confiança

# 27. Centro da imagem encontrada
if button_location:
    center = pyautogui.center(button_location)
    pyautogui.click(center)

# 28. Localizar por região específica
search_region = (100, 100, 500, 500) # X, Y, Width, Height
button = pyautogui.locateOnScreen('botao.png', region=search_region)
```

```
# 29. Capturar e salvar região para debug
pyautogui.screenshot('debug_region.png', region=(100, 100, 200, 200))
```

4. ALERTAS E MENSAGENS

python

```
# 30. Caixa de alerta
pyautogui.alert('Processo iniciado! Não mexa no computador.')

# 31. Caixa de confirmação (OK/Cancelar)
result = pyautogui.confirm('Deseja continuar?')
if result == 'OK':
    print("Usuário confirmou")
else:
    print("Usuário cancelou")

# 32. Prompt de entrada
nome = pyautogui.prompt('Digite seu nome:')
print(f"Nome digitado: {nome}")

# 33. Prompt de senha (oculta caracteres)
senha = pyautogui.password('Digite sua senha:', mask='*')
print(f"Senha digitada: {senha}")

# 34. Título personalizado para alertas
pyautogui.alert(text='Processo finalizado!', title='Sistema RPA', button='OK')
```

5. CONFIGURAÇÕES E CONTROLE

python

```
# 35. Tamanho da tela
screen_width, screen_height = pyautogui.size()
print(f"Resolução: {screen_width}x{screen_height}")

# 36. Verificar se coordenadas estão dentro da tela
pyautogui.onScreen(100, 200) # Retorna True/False

# 37. Desabilitar failsafe (PERIGOSO - usar com cautela)
pyautogui.FAILSAFE = False
```

```

# 38. Aumentar velocidade (reduzir pausas)
pyautogui.PAUSE = 0.1 # 0.1 segundos entre comandos

# 39. Obter informações da tela
print(f"Posição atual: {pyautogui.position()}")
print(f"Tamanho da tela: {pyautogui.size()}")

# 40. Esperar por imagem aparecer
try:
    location = pyautogui.locateOnScreen('imagem.png', timeout=10) # Espera 10
segundos
    pyautogui.click(location)
except pyautogui.ImageNotFoundException:
    print("Imagem não apareceu em 10 segundos")

```

❑ COMANDOS AVANÇADOS PARA RPA

FUNÇÕES PERSONALIZADAS PARA RPA

python

```

import pyautogui
import time
from datetime import datetime

# 41. Mover mouse com verificação de failsafe
def safe_move_to(x, y, duration=0.5):
    """Move o mouse com segurança"""
    if pyautogui.FAILSAFE and (x == 0 and y == 0):
        raise pyautogui.FailSafeException("Failsafe ativado!")
    pyautogui.moveTo(x, y, duration=duration)

# 42. Clicar com verificação de visibilidade
def click_if_visible(image_path, confidence=0.8, timeout=5):
    """Clica em imagem se estiver visível"""
    start_time = time.time()
    while time.time() - start_time < timeout:
        try:
            location = pyautogui.locateOnScreen(image_path, confidence=confidence)

```

```

        if location:
            center = pyautogui.center(location)
            pyautogui.click(center)
            return True
        except pyautogui.ImageNotFoundException:
            time.sleep(0.5)
    return False

# 43. Digitar com validação
def type_with_validation(text, interval=0.1, max_attempts=3):
    """Digita texto e valida se foi digitado corretamente"""
    for attempt in range(max_attempts):
        pyautogui.write(text, interval=interval)
        time.sleep(0.5)

        # Aqui você implementaria a validação
        # Por exemplo, copiar o texto e comparar
        pyautogui.hotkey('ctrl', 'a')
        pyautogui.hotkey('ctrl', 'c')
        # ... lógica de validação

        if True: # Substituir por validação real
            return True

        print(f"Tentativa {attempt + 1} falhou, tentando novamente...")
    return False

# 44. Esperar por mudança de cor
def wait_for_color(x, y, expected_color, timeout=10, interval=0.5):
    """Espera até que um pixel tenha a cor esperada"""
    start_time = time.time()
    while time.time() - start_time < timeout:
        current_color = pyautogui.pixel(x, y)
        if current_color == expected_color:
            return True
        time.sleep(interval)
    return False

# 45. Drag and drop inteligente
def smart_drag_drop(start_x, start_y, end_x, end_y):
    """Arrasta e solta com verificação"""
    pyautogui.moveTo(start_x, start_y)
    pyautogui.mouseDown()
    time.sleep(0.2)

```

```
pyautogui.moveTo(end_x, end_y, duration=0.5)
pyautogui.mouseUp()
```

GERENCIAMENTO DE JANELAS E APLICAÇÕES

python

```
# 46. Ativar janela por título (usando pygetwindow junto)
# Instale: pip install pygetwindow
```

```
import pygetwindow as gw
```

```
def activate_window(title_contains):
    """Ativa janela que contém o texto no título"""
    windows = gw.getWindowsWithTitle(title_contains)
    if windows:
        windows[0].activate()
        time.sleep(0.5)
        return True
    return False
```

```
# 47. Maximizar/minimizar janela
```

```
def maximize_window(title):
    windows = gw.getWindowsWithTitle(title)
    if windows:
        windows[0].maximize()
        return True
    return False
```

```
# 48. Obter informações da janela ativa
```

```
def get_active_window_info():
    active = gw.getActiveWindow()
    if active:
        return {
            'title': active.title,
            'size': active.size,
            'position': active.topleft
        }
    return None
```

UTILITÁRIOS PARA AUTOMAÇÃO

python


```
# 49. Logger com timestamp
def log_action(action, details=""):
    """Registra ação com timestamp"""
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    if details:
        print(f"[{timestamp}] {action}: {details}")
    else:
        print(f"[{timestamp}] {action}")

# 50. Monitor de posição do mouse (debug)
def mouse_position_monitor(interval=1, duration=10):
    """Monitora a posição do mouse por um tempo"""
    start_time = time.time()
    positions = []

    print("Iniciando monitoramento de mouse...")
    print("Mova o mouse para o canto superior esquerdo para parar.")

    try:
        while time.time() - start_time < duration:
            x, y = pyautogui.position()
            positions.append((x, y, time.time()))
            print(f"Posição: ({x}, {y})")
            time.sleep(interval)
    except KeyboardInterrupt:
        print("\nMonitoramento interrompido pelo usuário.")

    return positions
```

❑ EXEMPLO PRÁTICO COMPLETO PARA RPA

python

```
"""
EXEMPLO COMPLETO: Automação de Login e Exportação
Automação RPA usando apenas PyAutoGUI
"""

import pyautogui
import time
import logging
```

```

from datetime import datetime

# Configuração
pyautogui.FAILSAFE = True
pyautogui.PAUSE = 0.3

# Configurar logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

class RPAPyAutoGUI:
    def __init__(self):
        self.screen_width, self.screen_height = pyautogui.size()
        logger.info(f"Resolução da tela: {self.screen_width}x{self.screen_height}")

    def take_screenshot(self, name="screenshot"):
        """Tira screenshot e salva com timestamp"""
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"{name}_{timestamp}.png"
        pyautogui.screenshot(filename)
        logger.info(f"Screenshot salvo: {filename}")
        return filename

    def click_image(self, image_path, timeout=10, confidence=0.9):
        """Clica em uma imagem quando ela aparece"""
        logger.info(f"Procurando imagem: {image_path}")

        start_time = time.time()
        while time.time() - start_time < timeout:
            try:
                location = pyautogui.locateOnScreen(image_path, confidence=confidence)
                if location:
                    center = pyautogui.center(location)
                    pyautogui.click(center)
                    logger.info(f"Imagem encontrada e clicada em {center}")
                    return True
            except pyautogui.ImageNotFoundException:
                time.sleep(1)

        logger.warning(f"Imagem não encontrada em {timeout} segundos: {image_path}")
        return False

```

```
def fill_field(self, x, y, text, clear_first=True):
    """Preenche campo em coordenadas específicas"""
    pyautogui.click(x, y)
    if clear_first:
        pyautogui.hotkey('ctrl', 'a')
        pyautogui.press('backspace')
    pyautogui.write(text)
    logger.info(f"Campo preenchido: {text}")

def login_system(self, username, password):
    """Automação de login"""
    logger.info("Iniciando processo de login")

    # Abrir sistema (exemplo)
    pyautogui.hotkey('winleft', 'r')
    time.sleep(1)
    pyautogui.write('https://sistema.com')
    pyautogui.press('enter')
    time.sleep(3)

    # Preencher credenciais
    self.fill_field(300, 250, username) # Campo usuário
    pyautogui.press('tab')
    pyautogui.write(password) # Campo senha
    pyautogui.press('enter')

    # Verificar login
    time.sleep(2)
    self.take_screenshot("apos_login")

    logger.info("Login concluído")

def export_data(self, export_path):
    """Exporta dados do sistema"""
    logger.info("Iniciando exportação de dados")

    # Navegar para menu de exportação
    self.click_image('images/menu_exportar.png')
    time.sleep(1)

    # Selecionar formato
    pyautogui.click(400, 300) # Clique em Excel
```

```

time.sleep(0.5)

# Clicar em exportar
pyautogui.click(500, 400)
time.sleep(2)

# Salvar diálogo
pyautogui.write(export_path)
pyautogui.press('enter')

# Aguardar exportação
if self.wait_for_export_complete():
    logger.info("Exportação concluída com sucesso")
    return True
else:
    logger.error("Falha na exportação")
    return False

def wait_for_export_complete(self, timeout=30):
    """Aguarda até que a exportação esteja completa"""
    logger.info("Aguardando conclusão da exportação...")

    # Monitorar pixel que indica conclusão
    complete_color = (0, 128, 0) # Verde
    start_time = time.time()

    while time.time() - start_time < timeout:
        # Verificar cor do pixel de status
        if pyautogui.pixelMatchesColor(600, 500, complete_color, tolerance=10):
            logger.info("Exportação detectada como completa")
            return True

        # Verificar se há mensagem de erro
        if pyautogui.pixelMatchesColor(600, 500, (255, 0, 0), tolerance=10): #
Vermelho
            logger.error("Erro detectado durante exportação")
            return False

        time.sleep(1)

    logger.warning(f"Timeout após {timeout} segundos")
    return False

```

```

def run_full_process(self):
    """Executa processo RPA completo"""
    try:
        logger.info("=" * 50)
        logger.info("INICIANDO PROCESSO RPA")
        logger.info("=" * 50)

        # Notificar usuário
        pyautogui.alert(
            "O processo RPA será iniciado.\n"
            "Não use o mouse ou teclado durante a execução.\n"
            "Para abortar, mova o mouse para o canto superior esquerdo.",
            "Sistema RPA"
        )

        # Capturar estado inicial
        self.take_screenshot("inicio_processo")

        # Executar passos
        self.login_system("admin", "senha123")
        time.sleep(2)

        self.export_data("C:\\Exportados\\dados.xlsx")
        time.sleep(2)

        # Finalizar
        self.take_screenshot("fim_processo")

        pyautogui.alert("Processo RPA concluído com sucesso!", "Sistema RPA")
        logger.info("Processo concluído com sucesso")

    except pyautogui.FailSafeException:
        logger.warning("Processo abortado pelo usuário (failsafe)")
        pyautogui.alert("Processo RPA abortado pelo usuário", "Sistema RPA")

    except Exception as e:
        logger.error(f"Erro durante o processo: {str(e)}")
        self.take_screenshot("erro_processo")
        pyautogui.alert(f"Erro no processo RPA: {str(e)[:50]}", "Sistema RPA")

# Executar
if __name__ == "__main__":

```

```
rpa = RPAPyAutoGUI()  
rpa.run_full_process()
```

❑ RESUMO DOS 50 MAIS USADOS EM RPA

TOP 20 COMANDOS MAIS UTILIZADOS:

1. `pyautogui.position()` - Posição atual do mouse
2. `pyautogui.moveTo(x, y)` - Mover mouse para coordenadas
3. `pyautogui.click()` - Clicar com mouse
4. `pyautogui.write(text)` - Digitar texto
5. `pyautogui.press(key)` - Pressionar tecla
6. `pyautogui.hotkey(key1, key2)` - Atalhos de teclado
7. `pyautogui.screenshot()` - Capturar tela
8. `pyautogui.locateOnScreen(image)` - Encontrar imagem na tela
9. `pyautogui.size()` - Tamanho da tela
10. `pyautogui.alert(text)` - Caixa de alerta
11. `pyautogui.doubleClick()` - Duplo clique
12. `pyautogui.rightClick()` - Clique direito
13. `pyautogui.scroll(value)` - Rolar página
14. `pyautogui.dragTo(x, y)` - Arrastar mouse
15. `pyautogui.pixel(x, y)` - Cor do pixel
16. `pyautogui.keyDown(key)` / `keyUp(key)` - Segurar/soltar tecla
17. `pyautogui.confirm(text)` - Caixa de confirmação
18. `pyautogui.prompt(text)` - Caixa de entrada
19. `pyautogui.password(text)` - Caixa de senha
20. `pyautogui.PAUSE = valor` - Configurar pausa entre comandos

COMANDOS AVANÇADOS FREQUENTES:

21. `pyautogui.moveRel(dx, dy)` - Movimento relativo
22. `pyautogui.locateAllOnScreen(image)` - Todas as ocorrências
23. `pyautogui.center(box)` - Centro de uma região

- 24. `pyautogui.pixelMatchesColor()` - Comparar cor de pixel
- 25. `pyautogui.mouseDown()` / `mouseUp()` - Pressionar/soltar mouse
- 26. `pyautogui.hscroll(value)` - Scroll horizontal
- 27. `pyautogui.FAILSAFE = True/False` - Ativar/desativar segurança
- 28. `pyautogui.onScreen(x, y)` - Verificar se coordenada é válida
- 29. `pyautogui.tripleClick()` - Triplo clique
- 30. `pyautogui.middleClick()` - Clique do meio

PADRÕES DE AUTOMAÇÃO COMUNS:

python

```
# Padrão 1: Login automático
def auto_login():
    pyautogui.write('usuario')
    pyautogui.press('tab')
    pyautogui.write('senha')
    pyautogui.press('enter')

# Padrão 2: Navegação por atalhos
def navigate_system():
    pyautogui.hotkey('alt', 'f') # Menu Arquivo
    pyautogui.press('down', presses=3)
    pyautogui.press('enter')

# Padrão 3: Preenchimento de formulário
def fill_form(data):
    for field, value in data.items():
        pyautogui.write(value)
        pyautogui.press('tab')

# Padrão 4: Download de arquivo
def download_file():
    pyautogui.rightClick() # Clique direito no link
    pyautogui.press('down', presses=2) # Navega para "Salvar como"
    pyautogui.press('enter')
    time.sleep(1)
    pyautogui.write('arquivo.pdf')
    pyautogui.press('enter')
```

BOAS PRÁTICAS PARA RPA COM PYAUTOGUI:

1. **Sempre ative FAILSAFE** - Segurança primeiro!
2. **Use PAUSE entre comandos** - Evita sobrecarga
3. **Capture screenshots** - Para debug e documentação
4. **Implemente timeouts** - Evita loops infinitos
5. **Use coordenadas relativas** quando possível
6. **Combine com image recognition** para robustez
7. **Log todas as ações** - Para auditoria
8. **Teste em diferentes resoluções**
9. **Use try/except** para ImageNotFoundException
10. **Crie funções reutilizáveis** - Modularize seu código

TRUQUES AVANÇADOS:

python

```
# 1. Calibrar coordenadas dinamicamente
def get_center_of_screen():
    width, height = pyautogui.size()
    return width // 2, height // 2

# 2. Encontrar elemento por cor
def find_by_color(target_color, region=None):
    screenshot = pyautogui.screenshot(region=region)
    width, height = screenshot.size

    for x in range(width):
        for y in range(height):
            if screenshot.getpixel((x, y)) == target_color:
                return x, y
    return None

# 3. Automatizar arrastar arquivo
def drag_file_to_folder(file_coords, folder_coords):
    pyautogui.moveTo(file_coords)
    pyautogui.mouseDown()
    pyautogui.moveTo(folder_coords, duration=1)
    pyautogui.mouseUp()

# 4. Monitorar atividade
def monitor_activity(duration=60):
```



```
positions = []
for _ in range(duration):
    positions.append(pyautogui.position())
    time.sleep(1)
return positions
```

❑ CUIDADOS IMPORTANTES:

1. **PyAutoGUI controla SEU mouse e teclado** - Teste em ambiente seguro
2. **Coordenadas são absolutas** - Mude de máquina, mude as coordenadas
3. **Timing é crítico** - Use `time.sleep()` quando necessário
4. **Screenshots podem variar** - Resolução, tema, DPI afetam
5. **Alguns apps bloqueiam automação** - Anti-cheat, bancos, etc.

❑ EXERCÍCIO PRÁTICO:

Crie um script que:

1. Abre o Bloco de Notas
2. Digita um texto
3. Salva o arquivo
4. Fecha o programa

python

```
# Solução:
import pyautogui
import time

pyautogui.hotkey('winleft', 'r')
time.sleep(1)
pyautogui.write('notepad')
pyautogui.press('enter')
time.sleep(2)
pyautogui.write('Texto automático do RPA!')
pyautogui.hotkey('ctrl', 's')
time.sleep(1)
pyautogui.write('documento_auto.txt')
pyautogui.press('enter')
```

```
time.sleep(1)
pyautogui.hotkey('alt', 'f4')
```